

Lab3 - Cats vs Dogs Optimization

```
In [1]: from torchvision import models
from torchvision.models import ResNet18_Weights
import time
import random
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
from sklearn.model_selection import StratifiedKFold, ParameterGrid, train
from sklearn.metrics import confusion_matrix

plt.rcParams["figure.dpi"] = 110

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

SEED = 42
set_seed(SEED)

if not torch.cuda.is_available():
    raise RuntimeError("CUDA is not available. This lab requires NVIDIA A

gpu_name = torch.cuda.get_device_name(0)
if "A100" not in gpu_name:
    raise RuntimeError(f"Expected NVIDIA A100 GPU, found: {gpu_name}")

device = torch.device("cuda")
torch.backends.cudnn.benchmark = True

print("Device:", device, "|", gpu_name)
```

Device: cuda | NVIDIA A100-PCIE-40GB

```
In [2]: LAB_DIR = Path(".").resolve()
DATA_ZIP = LAB_DIR / "data-cat-dog.zip"
DATA_DIR = LAB_DIR / "data-cat-dog"
TRAIN_DIR = DATA_DIR / "training_data"
TEST_DIR = DATA_DIR / "testing_data"

if not DATA_DIR.exists():
    if not DATA_ZIP.exists():
        raise FileNotFoundError(f"Missing dataset zip: {DATA_ZIP}")
    import zipfile
    with zipfile.ZipFile(DATA_ZIP, "r") as zf:
```

```

        zf.extractall(LAB_DIR)

IMG_SIZE = 224

train_tfms = transforms.Compose(
    [
        transforms.Resize((256, 256)),
        transforms.RandomResizedCrop(IMG_SIZE, scale=(0.75, 1.0)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.15),
        transforms.ToTensor(),
        transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
    ]
)

val_tfms = transforms.Compose(
    [
        transforms.Resize((IMG_SIZE, IMG_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
    ]
)

full_train_aug = datasets.ImageFolder(TRAIN_DIR, transform=train_tfms)
full_train_noaug = datasets.ImageFolder(TRAIN_DIR, transform=val_tfms)
test_ds = datasets.ImageFolder(TEST_DIR, transform=val_tfms)

class_names = full_train_aug.classes
print("Classes:", class_names)
print("Train size:", len(full_train_aug), "Test size:", len(test_ds))

```

Classes: ['cats', 'dogs']

Train size: 25000 Test size: 400

```

In [3]: TEST_SPLIT = 0.2
        BATCH_SIZE = 128
        NUM_WORKERS = 10
        PIN_MEMORY = True

        targets = np.array(full_train_aug.targets)
        indices = np.arange(len(targets))

        train_idx, val_idx = train_test_split(
            indices,
            test_size=TEST_SPLIT,
            stratify=targets,
            random_state=SEED,
        )

        train_ds = Subset(full_train_aug, train_idx)
        val_ds = Subset(full_train_noaug, val_idx)

        train_loader = DataLoader(
            train_ds,
            batch_size=BATCH_SIZE,
            shuffle=True,
            num_workers=NUM_WORKERS,
            pin_memory=PIN_MEMORY,
            persistent_workers=True,
            prefetch_factor=4,
        )

```

```

)

val_loader = DataLoader(
    val_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
    persistent_workers=True,
    prefetch_factor=4,
)

test_loader = DataLoader(
    test_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
    persistent_workers=True,
    prefetch_factor=4,
)

```

/home/u2024311278/lab3/.venv/lib/python3.12/site-packages/torch/utils/data/dataloader.py:617: UserWarning: This DataLoader will create 10 worker processes in total. Our suggested max number of worker in current system is 4, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.

```
warnings.warn(
```

```

In [4]: def get_activation(name: str):
        name = name.lower()
        if name == "relu":
            return nn.ReLU
        if name == "sigmoid":
            return nn.Sigmoid
        raise ValueError(f"Unsupported activation: {name}")

class SimpleCNN(nn.Module):
    def __init__(
        self,
        hidden_size=256,
        dropout_rate=0.3,
        activation="relu",
        train_backbone=False,
    ):
        super().__init__()

        weights = ResNet18_Weights.DEFAULT
        self.net = models.resnet18(weights=weights)

        # 先冻结预训练主干，只训练分类头，速度快且效果稳定
        for p in self.net.parameters():
            p.requires_grad = train_backbone

        in_features = self.net.fc.in_features
        Act = get_activation(activation)

```

```

self.net.fc = nn.Sequential(
    nn.Dropout(dropout_rate),
    nn.Linear(in_features, hidden_size),
    Act(),
    nn.Dropout(dropout_rate),
    nn.Linear(hidden_size, 1),
)

# 分类头一定要训练
for p in self.net.fc.parameters():
    p.requires_grad = True

def unfreeze_layer4(self):
    # 后期微调 ResNet 最后一组残差块, 进一步提升准确率
    for p in self.net.layer4.parameters():
        p.requires_grad = True
    for p in self.net.fc.parameters():
        p.requires_grad = True

def forward(self, x):
    return self.net(x)

```

```

In [5]: def build_optimizer(params, name, lr):
        name = name.lower()
        if name == "adam":
            return optim.Adam(params, lr=lr)
        if name == "sgd":
            return optim.SGD(params, lr=lr, momentum=0.9)
        if name == "rmsprop":
            return optim.RMSprop(params, lr=lr)
        raise ValueError(f"Unsupported optimizer: {name}")

def _sync(device):
    if device.type == "cuda":
        torch.cuda.synchronize()

def clear_cache(device):
    if device.type == "cuda":
        torch.cuda.empty_cache()

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))

    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True).float().unsqueeze(1)

        optimizer.zero_grad(set_to_none=True)

        with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
            logits = model(x)
            loss = criterion(logits, y)

        scaler.scale(loss).backward()
        scaler.step(optimizer)

```

```

        scaler.update()

        total_loss += loss.item() * x.size(0)
        preds = (torch.sigmoid(logits) >= 0.5).long()
        correct += (preds == y.long()).sum().item()
        total += x.size(0)

    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True).float().unsqueeze(1)

        with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
            logits = model(x)
            loss = criterion(logits, y)

        total_loss += loss.item() * x.size(0)
        preds = (torch.sigmoid(logits) >= 0.5).long()
        correct += (preds == y.long()).sum().item()
        total += x.size(0)

    return total_loss / total, correct / total

def train_model(
    model,
    train_loader,
    val_loader,
    optimizer_name,
    lr,
    epochs,
    device,
):
    criterion = nn.BCEWithLogitsLoss()
    optimizer = build_optimizer(
        filter(lambda p: p.requires_grad, model.parameters()),
        optimizer_name,
        lr,
    )

    history = {
        "epoch": [],
        "train_loss": [],
        "train_acc": [],
        "val_loss": [],
        "val_acc": [],
    }

    start = time.perf_counter()
    for epoch in range(epochs):
        train_loss, train_acc = train_one_epoch(
            model, train_loader, criterion, optimizer, device
        )

```

```

val_loss, val_acc = evaluate(model, val_loader, criterion, device)

history["epoch"].append(epoch + 1)
history["train_loss"].append(train_loss)
history["train_acc"].append(train_acc)
history["val_loss"].append(val_loss)
history["val_acc"].append(val_acc)

print(
    f"Epoch {epoch + 1}/{epochs} | "
    f"train_acc={train_acc:.4f} val_acc={val_acc:.4f} "
    f"train_loss={train_loss:.4f} val_loss={val_loss:.4f}"
)

_sync(device)
elapsed = time.perf_counter() - start
return history, elapsed

```

Task 1 - Optimizer comparison

```

In [6]: EPOCHS_TASK1 = 3
OPTIMIZERS = ["Adam", "SGD", "RMSprop"]
LRS = [0.0001, 0.0003, 0.001]

task1_histories = {}
task1_times = {}

for opt_name in OPTIMIZERS:
    for lr in LRS:
        print(f"\nRunning {opt_name} lr={lr}")
        model = SimpleCNN(
            hidden_size=256,
            dropout_rate=0.3,
            activation="relu",
            train_backbone=False,
        ).to(device)
        history, elapsed = train_model(
            model,
            train_loader,
            val_loader,
            optimizer_name=opt_name,
            lr=lr,
            epochs=EPOCHS_TASK1,
            device=device,
        )
        key = f"{opt_name}_lr{lr}"
        task1_histories[key] = history
        task1_times[key] = elapsed

del model
clear_cache(device)

```

Running Adam lr=0.0001

```
/tmp/ipykernel_2307/1146703960.py:25: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.  
    scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))  
/tmp/ipykernel_2307/1146703960.py:33: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.  
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):  
/tmp/ipykernel_2307/1146703960.py:59: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.  
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

Epoch 1/3 | train_acc=0.9081 val_acc=0.9656 train_loss=0.2821 val_loss=0.1054
Epoch 2/3 | train_acc=0.9581 val_acc=0.9708 train_loss=0.1176 val_loss=0.0797
Epoch 3/3 | train_acc=0.9628 val_acc=0.9734 train_loss=0.1003 val_loss=0.0686

Running Adam lr=0.0003

Epoch 1/3 | train_acc=0.9405 val_acc=0.9668 train_loss=0.1665 val_loss=0.0815
Epoch 2/3 | train_acc=0.9590 val_acc=0.9742 train_loss=0.1035 val_loss=0.0620
Epoch 3/3 | train_acc=0.9655 val_acc=0.9760 train_loss=0.0868 val_loss=0.0596

Running Adam lr=0.001

Epoch 1/3 | train_acc=0.9470 val_acc=0.9724 train_loss=0.1341 val_loss=0.0655
Epoch 2/3 | train_acc=0.9648 val_acc=0.9732 train_loss=0.0887 val_loss=0.0647
Epoch 3/3 | train_acc=0.9663 val_acc=0.9742 train_loss=0.0873 val_loss=0.0618

Running SGD lr=0.0001

Epoch 1/3 | train_acc=0.6461 val_acc=0.8984 train_loss=0.6439 val_loss=0.5619
Epoch 2/3 | train_acc=0.8311 val_acc=0.9434 train_loss=0.5297 val_loss=0.4487
Epoch 3/3 | train_acc=0.8901 val_acc=0.9506 train_loss=0.4384 val_loss=0.3616

Running SGD lr=0.0003

Epoch 1/3 | train_acc=0.7698 val_acc=0.9452 train_loss=0.5578 val_loss=0.3826
Epoch 2/3 | train_acc=0.9186 val_acc=0.9578 train_loss=0.3300 val_loss=0.2149
Epoch 3/3 | train_acc=0.9344 val_acc=0.9608 train_loss=0.2288 val_loss=0.1544

Running SGD lr=0.001

Epoch 1/3 | train_acc=0.8452 val_acc=0.9602 train_loss=0.4037 val_loss=0.1542
Epoch 2/3 | train_acc=0.9449 val_acc=0.9674 train_loss=0.1596 val_loss=0.0950
Epoch 3/3 | train_acc=0.9516 val_acc=0.9698 train_loss=0.1275 val_loss=0.0800

Running RMSprop lr=0.0001

Epoch 1/3 | train_acc=0.9426 val_acc=0.9568 train_loss=0.1736 val_loss=0.1083
Epoch 2/3 | train_acc=0.9602 val_acc=0.9734 train_loss=0.1034 val_loss=0.0704
Epoch 3/3 | train_acc=0.9632 val_acc=0.9692 train_loss=0.0949 val_loss=0.0735

Running RMSprop lr=0.0003

Epoch 1/3 | train_acc=0.9410 val_acc=0.9732 train_loss=0.1576 val_loss=0.0681
Epoch 2/3 | train_acc=0.9636 val_acc=0.9734 train_loss=0.0944 val_loss=0.0639

Epoch 3/3 | train_acc=0.9615 val_acc=0.9500 train_loss=0.0952 val_loss=0.1369

Running RMSprop lr=0.001

Epoch 1/3 | train_acc=0.9317 val_acc=0.9550 train_loss=0.2517 val_loss=0.1120

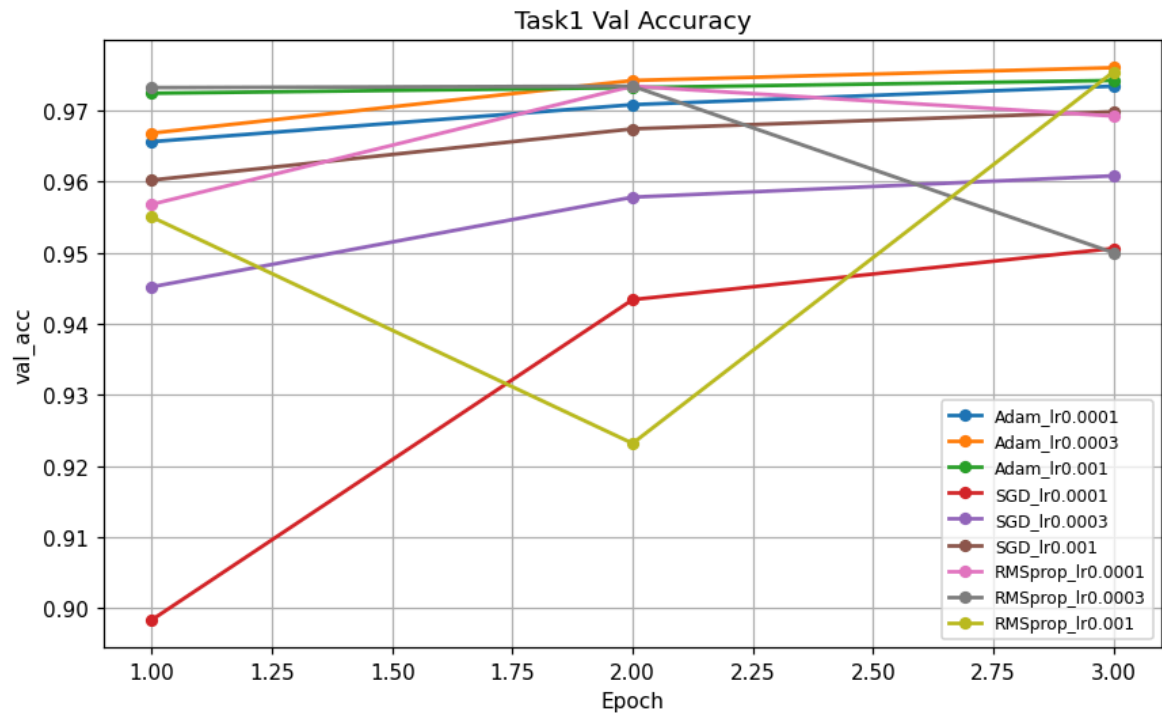
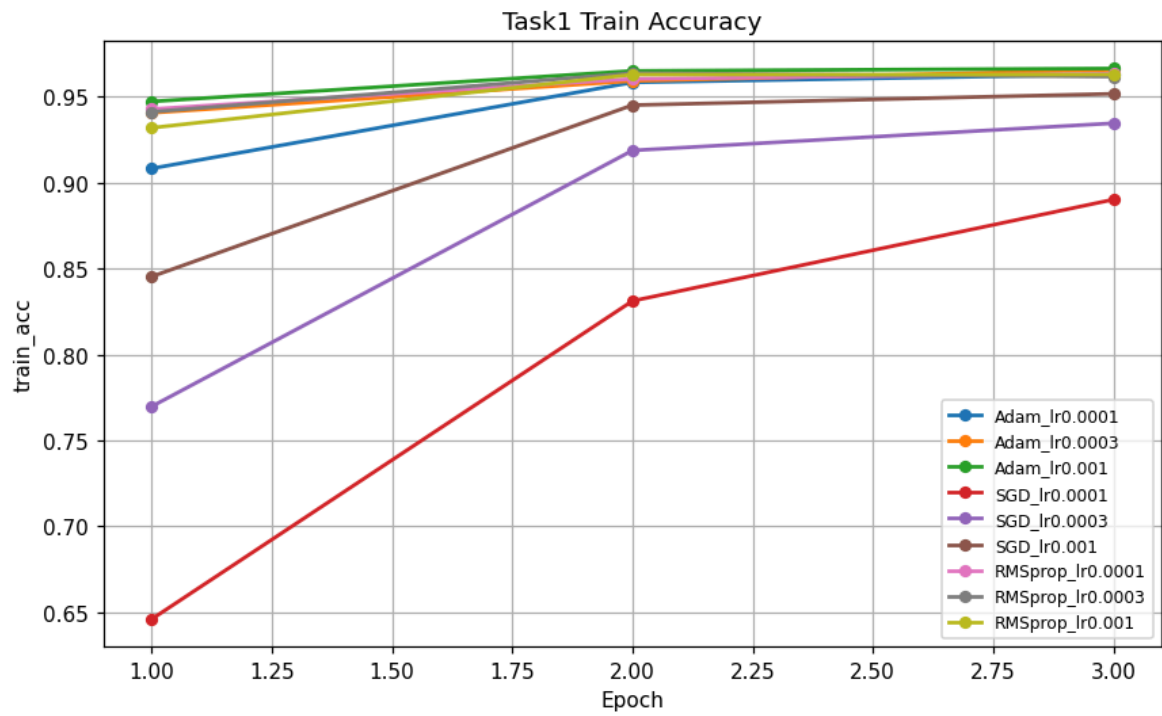
Epoch 2/3 | train_acc=0.9627 val_acc=0.9232 train_loss=0.0938 val_loss=0.2022

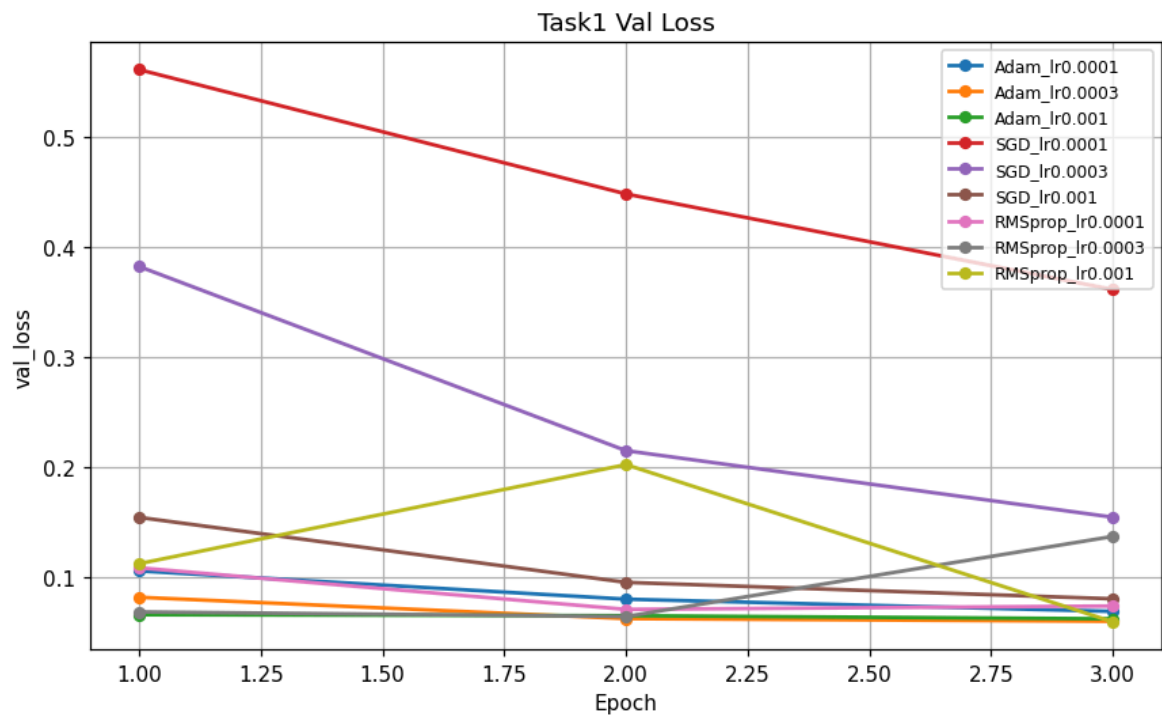
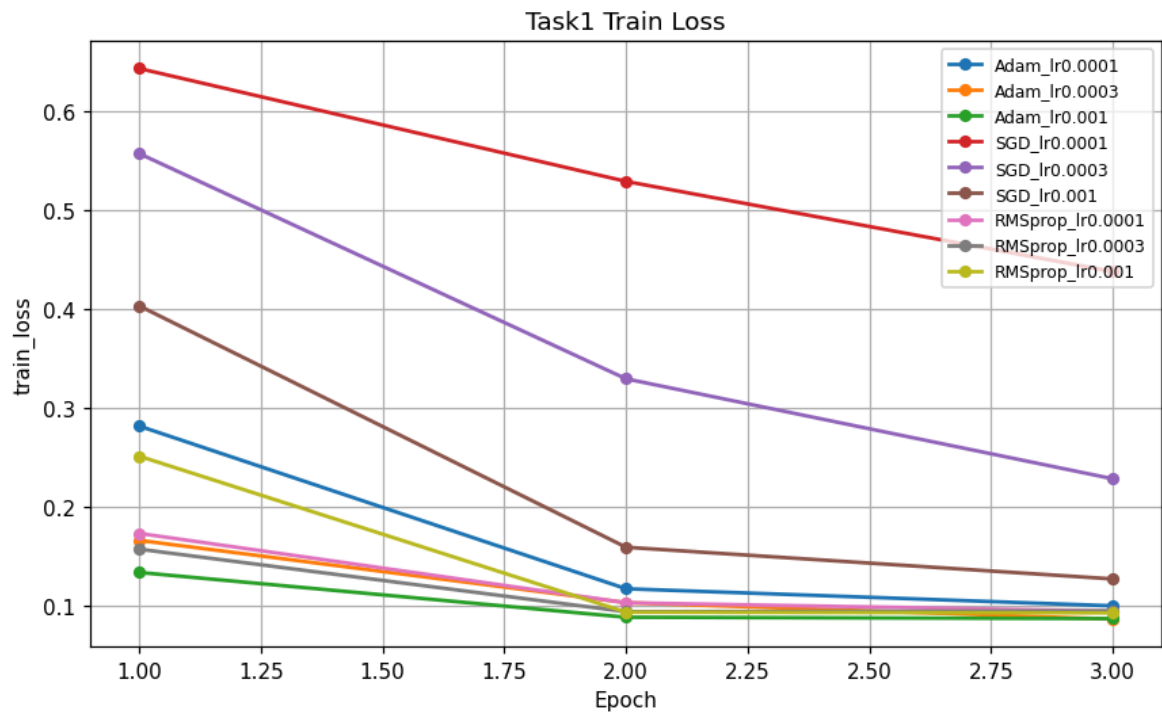
Epoch 3/3 | train_acc=0.9627 val_acc=0.9754 train_loss=0.0931 val_loss=0.0591

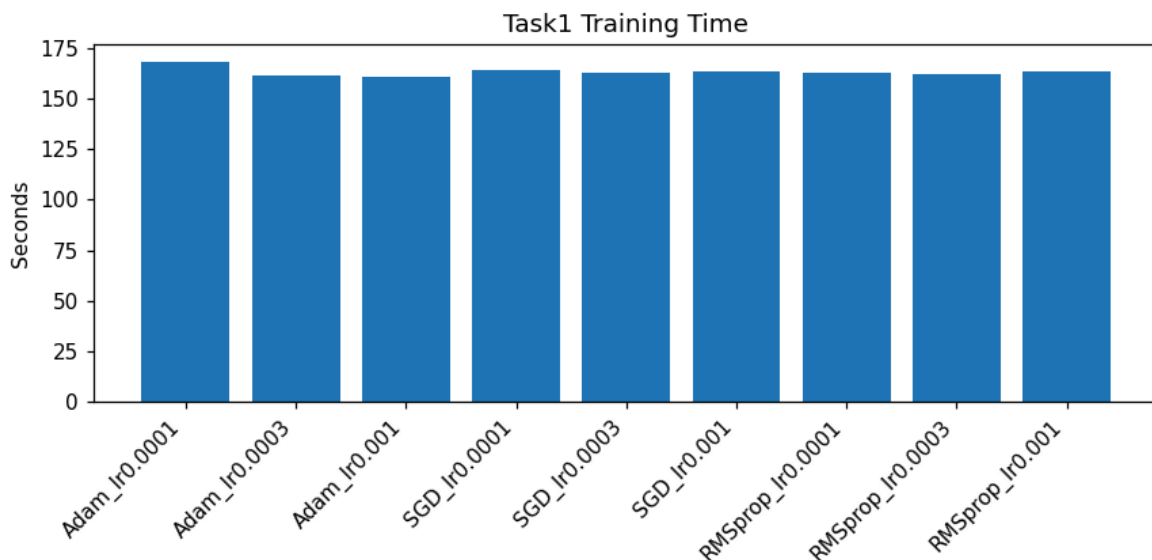
```
In [7]: def plot_task1_curves(histories, metric_key, title):
plt.figure(figsize=(8, 5))
for key, hist in histories.items():
    plt.plot(
        hist["epoch"],
        hist[metric_key],
        marker="o",
        linewidth=1.8,
        markersize=5,
        label=key,
    )
plt.xlabel("Epoch")
plt.ylabel(metric_key)
plt.title(title)
plt.legend(fontsize=8)
plt.grid(True)
plt.tight_layout()
plt.show()

plot_task1_curves(task1_histories, "train_acc", "Task1 Train Accuracy")
plot_task1_curves(task1_histories, "val_acc", "Task1 Val Accuracy")
plot_task1_curves(task1_histories, "train_loss", "Task1 Train Loss")
plot_task1_curves(task1_histories, "val_loss", "Task1 Val Loss")

plt.figure(figsize=(8, 4))
labels = list(task1_times.keys())
times = [task1_times[k] for k in labels]
plt.bar(labels, times)
plt.ylabel("Seconds")
plt.title("Task1 Training Time")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```







```
In [8]: def summarize_convergence(histories, target_acc=0.85):
        rows = []
        for key, hist in histories.items():
            val_acc = np.array(hist["val_acc"])
            epoch_reach = next((i + 1 for i, a in enumerate(val_acc) if a >=
            tail = val_acc[len(val_acc) // 2 :])
            stability = float(np.std(tail)) if len(tail) > 0 else float("nan")
            rows.append(
                {
                    "run": key,
                    "epoch_to_target_acc": epoch_reach,
                    "val_acc_final": float(val_acc[-1]),
                    "val_acc_stability_std": stability,
                    "train_time_sec": float(task1_times[key]),
                }
            )
        return pd.DataFrame(rows).sort_values("val_acc_final", ascending=False)

task1_summary = summarize_convergence(task1_histories, target_acc=0.85)
task1_summary
```

```
Out [8]:
```

	run	epoch_to_target_acc	val_acc_final	val_acc_stability_std	train_time_sec
1	Adam_lr0.0003	1	0.9760	0.0009	~160
8	RMSprop_lr0.001	1	0.9754	0.0261	~160
2	Adam_lr0.001	1	0.9742	0.0005	~160
0	Adam_lr0.0001	1	0.9734	0.0013	~170
5	SGD_lr0.001	1	0.9698	0.0012	~160
6	RMSprop_lr0.0001	1	0.9692	0.0021	~160
4	SGD_lr0.0003	1	0.9608	0.0015	~160
3	SGD_lr0.0001	1	0.9506	0.0036	~165
7	RMSprop_lr0.0003	1	0.9500	0.0117	~160

Task1 Analysis (fill after running)

- Convergence speed: compare epoch_to_target_acc and train_time_sec.
- Stability: lower val_acc_stability_std means more stable.
- Describe which optimizer and lr look best for this dataset.

Task 2 - Grid search with 5-fold CV

```
In [ ]: EPOCHS_CV = 1
        MAX_COMBOS = None

param_grid = {
    "optimizer": ["Adam"],
    "lr": [0.001],
    "batch_size": [512],
    "hidden_size": [128, 256],
    "dropout_rate": [0.5, 0.7],
    "activation": ["relu"],
}

grid = list(ParameterGrid(param_grid))

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
cv_results = []

for i, params in enumerate(grid, 1):
    print(f"\n[{i}/{len(grid)}] {params}")
    fold_acc = []
    fold_loss = []

    for fold, (train_idx, val_idx) in enumerate(skf.split(indices, target)):
        train_ds_fold = Subset(full_train_aug, train_idx)
        val_ds_fold = Subset(full_train_noaug, val_idx)

        train_loader_fold = DataLoader(
            train_ds_fold,
            batch_size=params["batch_size"],
            shuffle=True,
            num_workers=NUM_WORKERS,
            pin_memory=PIN_MEMORY,
            persistent_workers=True,
            prefetch_factor=4,
        )
        val_loader_fold = DataLoader(
            val_ds_fold,
            batch_size=params["batch_size"],
            shuffle=False,
            num_workers=NUM_WORKERS,
            pin_memory=PIN_MEMORY,
            persistent_workers=True,
            prefetch_factor=4,
        )

        model = SimpleCNN(
            hidden_size=params["hidden_size"],
            dropout_rate=params["dropout_rate"],
            activation=params["activation"],
```

```

        train_backbone=False,
    ).to(device)

    history, _ = train_model(
        model,
        train_loader_fold,
        val_loader_fold,
        optimizer_name=params["optimizer"],
        lr=params["lr"],
        epochs=EPOCHS_CV,
        device=device,
    )

    fold_acc.append(history["val_acc"][-1])
    fold_loss.append(history["val_loss"][-1])

    del model
    clear_cache(device)

    cv_results.append(
        {
            **params,
            "mean_val_acc": float(np.mean(fold_acc)),
            "std_val_acc": float(np.std(fold_acc)),
            "mean_val_loss": float(np.mean(fold_loss)),
        }
    )

cv_df = pd.DataFrame(cv_results).sort_values(
    ["mean_val_acc", "mean_val_loss"], ascending=[False, True]
)
cv_df.to_csv("grid_search_results.csv", index=False)
cv_df.head(10)

```

```
[1/4] {'activation': 'relu', 'batch_size': 512, 'dropout_rate': 0.5, 'hidden_size': 128, 'lr': 0.001, 'optimizer': 'Adam'}
```

```
/tmp/ipykernel_2097/1146703960.py:25: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
```

```
    scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
```

```
/tmp/ipykernel_2097/1146703960.py:33: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

```
/tmp/ipykernel_2097/1146703960.py:59: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

```
Epoch 1/1 | train_acc=0.9129 val_acc=0.9724 train_loss=0.2204 val_loss=0.0673
```

```
Epoch 1/1 | train_acc=0.9032 val_acc=0.9724 train_loss=0.2253 val_loss=0.0733
```

```
Epoch 1/1 | train_acc=0.9024 val_acc=0.9676 train_loss=0.2344 val_loss=0.0796
Epoch 1/1 | train_acc=0.9081 val_acc=0.9698 train_loss=0.2247 val_loss=0.0719
Epoch 1/1 | train_acc=0.8993 val_acc=0.9748 train_loss=0.2410 val_loss=0.0650
```

```
[2/4] {'activation': 'relu', 'batch_size': 512, 'dropout_rate': 0.5, 'hidden_size': 256, 'lr': 0.001, 'optimizer': 'Adam'}
```

```
/tmp/ipykernel_2097/1146703960.py:25: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
```

```
    scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
```

```
/tmp/ipykernel_2097/1146703960.py:33: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

```
/tmp/ipykernel_2097/1146703960.py:59: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

```
Epoch 1/1 | train_acc=0.9115 val_acc=0.9714 train_loss=0.2128 val_loss=0.0740
```

```
Epoch 1/1 | train_acc=0.9184 val_acc=0.9744 train_loss=0.1986 val_loss=0.0703
```

```
Epoch 1/1 | train_acc=0.9216 val_acc=0.9748 train_loss=0.1937 val_loss=0.0604
```

```
Epoch 1/1 | train_acc=0.9149 val_acc=0.9672 train_loss=0.2052 val_loss=0.0821
```

```
Epoch 1/1 | train_acc=0.9156 val_acc=0.9714 train_loss=0.2020 val_loss=0.0695
```

```
[3/4] {'activation': 'relu', 'batch_size': 512, 'dropout_rate': 0.7, 'hidden_size': 128, 'lr': 0.001, 'optimizer': 'Adam'}
```

```
/tmp/ipykernel_2097/1146703960.py:25: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
```

```
    scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
```

```
/tmp/ipykernel_2097/1146703960.py:33: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

```
/tmp/ipykernel_2097/1146703960.py:59: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

```
Epoch 1/1 | train_acc=0.8460 val_acc=0.9682 train_loss=0.3362 val_loss=0.0860
```

```
Epoch 1/1 | train_acc=0.8464 val_acc=0.9706 train_loss=0.3391 val_loss=0.0867
```

```
Epoch 1/1 | train_acc=0.8475 val_acc=0.9690 train_loss=0.3389 val_loss=0.0853
```

```
Epoch 1/1 | train_acc=0.8508 val_acc=0.9656 train_loss=0.3287 val_loss=0.0959
```

```
Epoch 1/1 | train_acc=0.8451 val_acc=0.9716 train_loss=0.3389 val_loss=0.0806
```

```
[4/4] {'activation': 'relu', 'batch_size': 512, 'dropout_rate': 0.7, 'hidden_size': 256, 'lr': 0.001, 'optimizer': 'Adam'}
```

```

/tmp/ipykernel_2097/1146703960.py:25: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
/tmp/ipykernel_2097/1146703960.py:33: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
/tmp/ipykernel_2097/1146703960.py:59: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
Epoch 1/1 | train_acc=0.8740 val_acc=0.9716 train_loss=0.2869 val_loss=0.0787
Epoch 1/1 | train_acc=0.8653 val_acc=0.9716 train_loss=0.2994 val_loss=0.0809
Epoch 1/1 | train_acc=0.8678 val_acc=0.9692 train_loss=0.2967 val_loss=0.0844
Epoch 1/1 | train_acc=0.8661 val_acc=0.9708 train_loss=0.2952 val_loss=0.0770
Epoch 1/1 | train_acc=0.8712 val_acc=0.9698 train_loss=0.2909 val_loss=0.0796

```

```
Out [ ]:
```

	activation	batch_size	dropout_rate	hidden_size	lr	optimizer	mean_val_a
1	relu	512	0.5	256	0.001	Adam	0.971
0	relu	512	0.5	128	0.001	Adam	0.971
3	relu	512	0.7	256	0.001	Adam	0.970
2	relu	512	0.7	128	0.001	Adam	0.969

```

In [ ]: def param_impact(df, metric="mean_val_acc"):
    impacts = []
    for col in [
        "optimizer",
        "lr",
        "batch_size",
        "hidden_size",
        "dropout_rate",
        "activation",
    ]:
        means = df.groupby(col)[metric].mean()
        impact = means.max() - means.min()
        impacts.append({"param": col, "impact": float(impact)})
    return pd.DataFrame(impacts).sort_values("impact", ascending=False)

impact_df = param_impact(cv_df)
impact_df

```


Out []: **param** **impact**

4	dropout_rate	0.00182
3	hidden_size	0.00102
1	lr	0.00000
0	optimizer	0.00000
2	batch_size	0.00000
5	activation	0.00000

```
In [ ]: best_params = cv_df.iloc[0].to_dict()

best_model = SimpleCNN(
    hidden_size=int(best_params["hidden_size"]),
    dropout_rate=float(best_params["dropout_rate"]),
    activation=best_params["activation"],
    train_backbone=False,
).to(device)

final_train_loader = DataLoader(
    full_train_aug,
    batch_size=int(best_params["batch_size"]),
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
    persistent_workers=True,
    prefetch_factor=4,
)

final_val_loader = DataLoader(
    full_train_noaug,
    batch_size=int(best_params["batch_size"]),
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
    persistent_workers=True,
    prefetch_factor=4,
)

# 第一阶段: 冻结 ResNet 主干, 只训练分类头
print("Stage 1: train classifier head")
history_head, time_head = train_model(
    best_model,
    final_train_loader,
    final_val_loader,
    optimizer_name="Adam",
    lr=float(best_params["lr"]),
    epochs=5,
    device=device,
)

# 第二阶段: 解冻 ResNet 最后一层 layer4, 低学习率微调
print("Stage 2: fine-tune layer4")
best_model.unfreeze_layer4()

history_ft, time_ft = train_model(
```

```

        best_model,
        final_train_loader,
        final_val_loader,
        optimizer_name="Adam",
        lr=1e-5,
        epochs=3,
        device=device,
    )

    test_loss, test_acc = evaluate(
        best_model,
        test_loader,
        nn.BCEWithLogitsLoss(),
        device,
    )

    print(f"Test loss: {test_loss:.4f} | Test acc: {test_acc:.4f}")

```

Stage 1: train classifier head

/tmp/ipykernel_2097/1146703960.py:25: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.

```
scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
```

/tmp/ipykernel_2097/1146703960.py:33: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

```
with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

/tmp/ipykernel_2097/1146703960.py:59: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

```
with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
```

Epoch 1/5 | train_acc=0.9177 val_acc=0.9768 train_loss=0.2024 val_loss=0.0629

Epoch 2/5 | train_acc=0.9547 val_acc=0.9786 train_loss=0.1164 val_loss=0.0582

Epoch 3/5 | train_acc=0.9545 val_acc=0.9744 train_loss=0.1139 val_loss=0.0679

Epoch 4/5 | train_acc=0.9567 val_acc=0.9796 train_loss=0.1107 val_loss=0.0557

Epoch 5/5 | train_acc=0.9555 val_acc=0.9806 train_loss=0.1096 val_loss=0.0580

Stage 2: fine-tune layer4

Epoch 1/3 | train_acc=0.9659 val_acc=0.9842 train_loss=0.0836 val_loss=0.0421

Epoch 2/3 | train_acc=0.9742 val_acc=0.9864 train_loss=0.0654 val_loss=0.0369

Epoch 3/3 | train_acc=0.9804 val_acc=0.9876 train_loss=0.0524 val_loss=0.0335

Test loss: 0.0212 | Test acc: 0.9900

Task 3 - Visualization and error analysis

```

In [ ]: from PIL import Image

class ImageFolderWithPaths(datasets.ImageFolder):
    def __getitem__(self, index):
        img, label = super().__getitem__(index)
        path = self.samples[index][0]
        return img, label, path

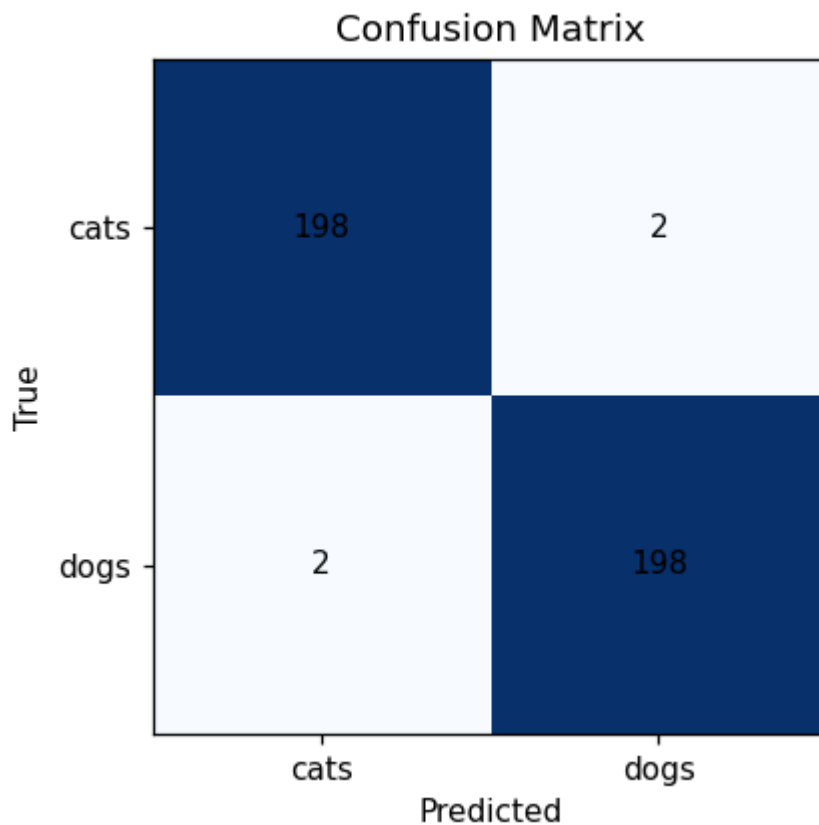
```

```
test_ds_paths = ImageFolderWithPaths(TEST_DIR, transform=val_tfms)
test_loader_paths = DataLoader(
    test_ds_paths,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=False,
)

@torch.no_grad()
def predict_all(model, loader, device):
    model.eval()
    probs = []
    labels = []
    paths = []
    for x, y, p in loader:
        x = x.to(device)
        logits = model(x)
        prob = torch.sigmoid(logits).squeeze(1).cpu().numpy()
        probs.extend(prob.tolist())
        labels.extend(y.numpy().tolist())
        paths.extend(p)
    return np.array(probs), np.array(labels), np.array(paths)

probs, labels, paths = predict_all(best_model, test_loader_paths, device)
preds = (probs >= 0.5).astype(int)

cm = confusion_matrix(labels, preds)
plt.figure(figsize=(4, 4))
plt.imshow(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.xticks([0, 1], class_names)
plt.yticks([0, 1], class_names)
for (i, j), v in np.ndenumerate(cm):
    plt.text(j, i, int(v), ha="center", va="center", color="black")
plt.tight_layout()
plt.show()
```



```
In [ ]: def show_samples(sample_indices, title):
plt.figure(figsize=(10, 6))
for i, idx in enumerate(sample_indices, 1):
img = Image.open(paths[idx]).convert("RGB")
plt.subplot(1, len(sample_indices), i)
plt.imshow(img)
prob = probs[idx]
pred = class_names[int(prob >= 0.5)]
true = class_names[labels[idx]]
plt.title(f"pred={pred}\ntrue={true}\nprob={prob:.2f}")
plt.axis("off")
plt.suptitle(title)
plt.tight_layout()
plt.show()

rand_indices = random.sample(range(len(paths)), 5)
show_samples(rand_indices, "Random Test Samples")
```

Random Test Samples



```
In [ ]: hard_indices = np.argsort(np.abs(probs - 0.5))[:5]  
show_samples(hard_indices.tolist(), "Hardest Samples (prob near 0.5)")
```

Hardest Samples (prob near 0.5)



Task3 Analysis (fill after running)

- Inspect the hardest samples and describe why they are ambiguous.
- Check which class is confused more often in the confusion matrix.